

Agentic Research RAG System

A Self-Directed AI Research Agent using Local LLM Reasoning, Live Web Search, and Iterative Self-Reflection

Prepared by: Vamshikadevaraj

Environment: Google Colab (T4 GPU)

Model: Qwen2.5-7B-Instruct (4-bit quantized)

1. What Problem Does This Solve?

A search engine like Google returns a ranked list of links. It does not read the pages, does not judge whether a source is trustworthy, and does not produce an answer — the human has to open every link and synthesize the information themselves.

This project builds an AI agent that performs the entire research process end-to-end and autonomously:

- Decides what to search for, based on understanding the question
- Retrieves live results from the web (no pre-baked training data, no API key)
- Reads and extracts the actual article text from each page
- Judges each source's relevance and reliability before trusting it
- Writes a single coherent, cited answer from multiple sources
- Checks its own answer for completeness, and searches again if it falls short

This pattern is called Agentic RAG (Retrieval-Augmented Generation). Standard RAG performs one retrieval + one generation step. "Agentic" means the LLM itself is making autonomous decisions at every stage — what to search, what to trust, when to stop — rather than following one fixed pipeline.

2. System Architecture

The agent runs a loop of five stages, repeating until it judges its own answer sufficient (or a maximum iteration cap is reached):

Stage	What It Does
1. Plan	The LLM reads the research question and generates 2-4 targeted search queries — it decides the search strategy itself, rather than searching the raw question verbatim.
2. Search & Fetch	Each query is run through DuckDuckGo (via the ddgs library). For every result URL, the actual page is downloaded and its clean article text is extracted (via trafilatura), stripping ads, navigation, and boilerplate.
3. Validate	The LLM reads each extracted page and scores it 1-5 for relevance and reliability to the original question. Sources scoring below a threshold (default 3) are discarded.
4. Synthesize	The LLM writes one coherent answer using only the validated evidence, citing each source inline as [1], [2], etc.

5. Reflect	The LLM re-reads its own draft answer and judges whether it fully answers the question. If gaps remain, it generates new follow-up search queries and the loop repeats (up to max_iterations, default 3). If the answer is sufficient, the agent stops.
------------	---

3. Technology Stack

Component	Tool / Library
Reasoning engine	Qwen2.5-7B-Instruct — an open-source instruction-tuned LLM, run entirely locally
Quantization	4-bit (NF4) via bitsandbytes — shrinks the model to ~5-6 GB VRAM so it fits a free-tier T4 GPU
Model loading	transformers + accelerate (Hugging Face)
Live web search	ddgs (DuckDuckGo Search) — free, no API key required
Page text extraction	trafilatura — downloads a page and extracts clean readable article text, removing ads/navigation/boilerplate
Compute environment	Google Colab, free-tier T4 GPU (16 GB VRAM)

Key point for the examiner: no proprietary API (OpenAI, Anthropic, etc.) is used anywhere in this system. The reasoning model, the search, and the extraction are all free and open-source, running entirely inside the Colab session.

4. Why This Is Not "Just Google"

Capability	Google Search vs. This Agent
Decides search strategy	Google: No (user types the query) — Agent: Yes, the LLM plans its own queries
Reads page content	Google: No (shows snippets only) — Agent: Yes, full article text is extracted
Judges source quality	Google: Ranks by algorithm/SEO — Agent: LLM explicitly scores relevance & reliability per source
Produces an answer	Google: No, only links — Agent: Yes, a single written, cited answer
Self-checks its own output	Google: N/A — Agent: Yes, reflection step re-evaluates and can trigger new searches

5. Engineering Challenges Resolved

Building this surfaced real dependency and API issues, which is worth mentioning to an examiner as evidence of hands-on debugging, not just running a tutorial:

- **Package rename:** the search library was renamed from duckduckgo-search to ddgs mid-development; all imports and install commands had to be updated to the new package name.
- **Breaking API change in trafilatura 2.x:** `fetch_url()` no longer accepts a timeout keyword argument directly. Fixed by creating a `trafilatura Config` object (`use_config()`) and setting `DOWNLOAD_TIMEOUT` through it instead.
- **Graceful failure handling:** some sites return HTTP 403 (blocking automated requests) or malformed responses. The fetch function catches these exceptions and returns an empty string instead of crashing, so the agent simply skips unreachable sources and continues.
- **Typo-driven install failure:** an initial dependency line had a misspelled package (`bitsandby` instead of `bitsandbytes`, and an invalid `ddgs-search` name), causing a silent partial install that only surfaced later as a `ModuleNotFoundError`. Diagnosed by isolating each import individually.

6. Verified Test Run

Question posed to the agent:

"What are the current best practices for reducing hallucinations in retrieval-augmented generation systems?"

Result:

- Iteration 1: planned 3 search queries
- Fetched 8 pages of live web content successfully (a few sources 403'd/blocked scrapers and were correctly skipped)
- All 8 fetched sources validated as relevant (score ≥ 3 out of 5)
- Synthesized a structured, inline-cited answer covering 7 distinct mitigation strategies (external grounding, retrieval tuning, fine-tuning, answer stabilization, fidelity checks, iterative refinement, hybrid approaches)
- Reflection step judged the answer complete — agent stopped after a single iteration, avoiding unnecessary extra search rounds

This confirms every stage of the pipeline — plan, search, fetch, validate, synthesize, reflect — executed correctly end-to-end on a real, unseen question with live internet data.

7. Anticipated Examiner Questions

Q: Why use a local open-source model instead of an API like GPT-4?

Cost and independence: a local 4-bit quantized model runs entirely for free on a Colab T4 GPU with no API key, no per-token billing, and no dependency on an external service being available. It also demonstrates lower-level understanding of quantization and model deployment, not just API calls.

Q: What does 4-bit quantization actually do?

It compresses each model weight from its normal 16/32-bit floating point representation down to 4 bits, drastically cutting memory usage (this 7-billion-parameter model drops to roughly 5-6 GB of VRAM) with only a small, generally acceptable accuracy trade-off — this is what makes a 7B model runnable on a free-tier GPU.

Q: What is the difference between this and standard RAG?

Standard RAG is a single retrieve-then-generate step. This system is agentic: the LLM plans its own search queries, judges source quality itself, and reflects on its own answer to decide whether to search again — all autonomous decisions, not a fixed one-shot pipeline.

Q: How does the system avoid hallucinating or making things up?

The synthesis step is explicitly instructed to answer using only the retrieved evidence and to say so if evidence is incomplete or conflicting. The validation step also filters out low-relevance sources before they ever reach the synthesis stage, reducing the chance of irrelevant or unreliable material influencing the answer.

Q: What happens if a website blocks the request?

The fetch function is wrapped in error handling: if a page returns a non-200 response (e.g. 403 Forbidden) or fails to parse, it returns an empty string instead of crashing, and that source is simply excluded from the evidence set. The agent continues with whatever sources it could successfully reach.

Q: How many times can the agent search before giving up?

A `max_iterations` parameter (default 3) caps the reflect-and-retry loop, preventing infinite searching. In practice, the tested run stopped after just 1 iteration because the agent judged its own answer sufficient.

Q: Could this be extended for a real product?

Yes — the notes in the notebook itself point out two concrete upgrades: swapping DuckDuckGo for a paid, more robust search API (Tavily, SerpAPI, Bing) for production reliability, and swapping the model for a stronger one (e.g. Llama-3.1-8B-Instruct) if higher reasoning quality is needed.

8. Possible Future Work

- Ablation study on `relevance_threshold` and `max_iterations`, measuring answer quality vs. runtime cost — consistent with the controlled-ablation methodology used in the two prior academic papers on hidden-layer depth.
- Add a proper User-Agent header to page requests to reduce 403 blocks from sites that reject default scraper headers.
- Persist results to JSON/CSV for logging and later analysis instead of only printing to console.